



LAN TRUYỀN NGƯỢC THEO THỜI GIAN TRONG MẠNG NƠ-RON HỒI TIẾP

(Recurrent Neural Network's Backpropagation Through Time (RNN BPTT))

Môn học: Cơ sở Toán cho Khoa Học Máy Tính (CO5263)

Báo cáo bài tập lớn.

Giảng viên hướng dẫn:

TS. Nguyễn An Khương

TS. Trần Tuấn Anh

Nhóm học viên thực hiện:

Đỗ Thanh Thái 2480860

Nguyễn Minh Quân 2212804

Nguyễn Quỳnh Như 2212472

Nguyễn Tấn Phú 2480859

Phạm Lê Ngọc Sơn 2470741

Phạm Duy Tùng 2470753

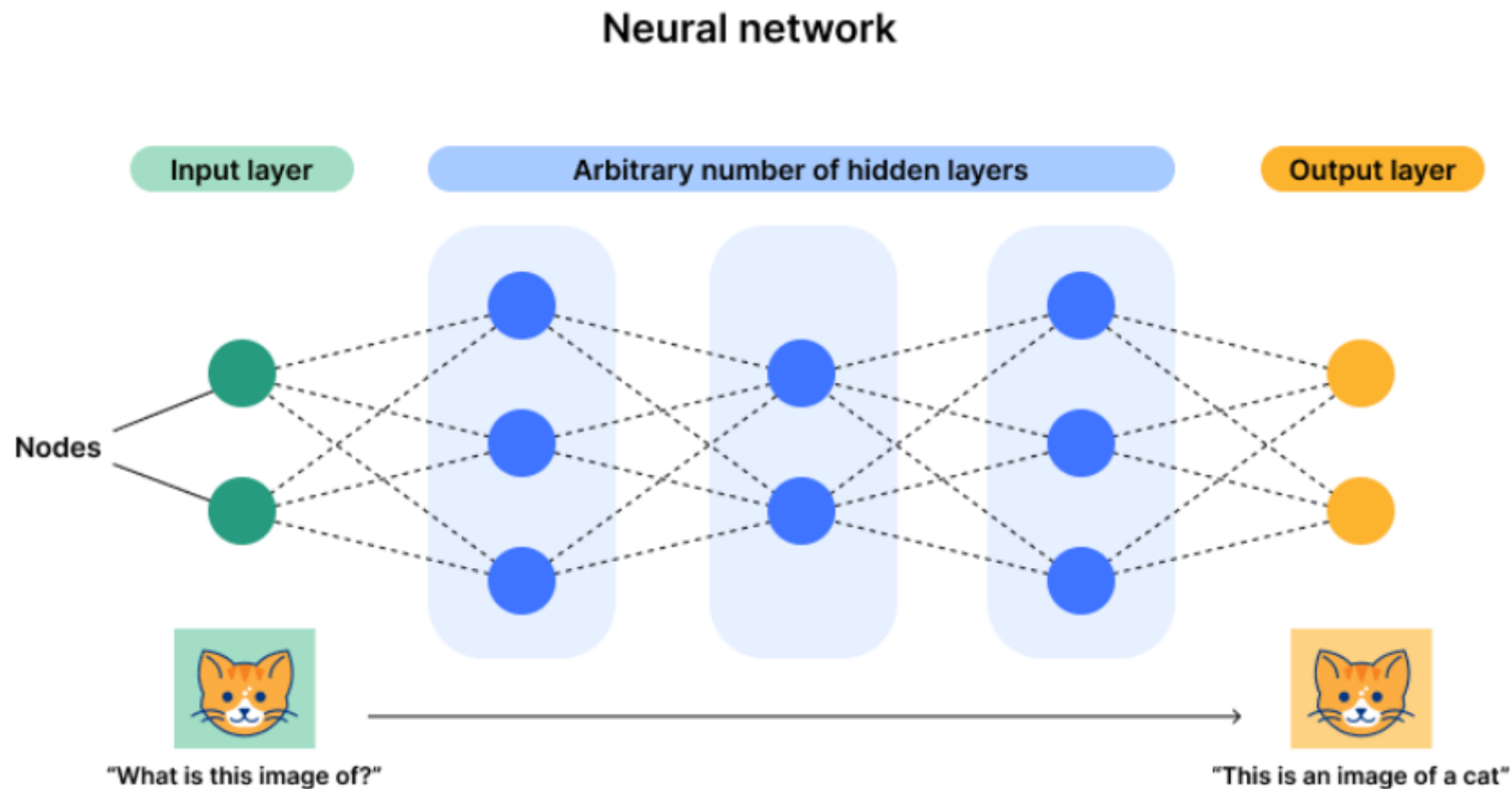
Danh mục trình bày

Giới thiệu Mạng nơ-ron hồi tiếp (RNN)

Nền tảng Toán học của Lan truyền ngược qua thời gian

Lan truyền ngược qua thời gian (Backpropagation through time - BPTT)

- Tính toán đầy đủ (Full Computation) BPTT
- Giải pháp và Các phương pháp huấn luyện khác
 - Cắt cố định (Fixed Truncation)
 - Cắt ngẫu nhiên (Randomized Truncation)
- Các ứng dụng của BPTT
- Hạn chế, Xu hướng nghiên cứu và Định hướng tương lai



Hình 1: Mô hình Neural Network đơn giản. [1]

Hạn chế của Neural Network truyền thống với dữ liệu tuần tự theo thời gian

Neural Network truyền thống chỉ có thể xử lý thông tin theo một chiều từ Input $\rightarrow$ Output. Công thức chung:

$$\mathbf{h}^{(l)} = \sigma(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \quad (1.1)$$

Trong đó:

- $\mathbf{h}^{(l)}$ là vector output tầng l .
- $\mathbf{W}^{(l)}$, $\mathbf{b}^{(l)}$ là ma trận trọng số và bias.
- σ là hàm kích hoạt (ReLU, tanh, sigmoid...).

Giả sử với chuỗi input:

$$\mathbf{x}_1, \mathbf{x}_2, \mathbf{x}_3, \dots, \mathbf{x}_T \quad (1.2)$$

Với Neural Network truyền thống:

$$\mathbf{y}_t = f(\mathbf{x}_t) \quad (1.3)$$

Nhưng thực tế cần xử lý tuần tự:

$$\mathbf{y}_t = f(\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_t) \quad (1.4)$$

Recurrent Neural Network

Cập nhật trạng thái ẩn:

$$\mathbf{H}_t = \varphi(\mathbf{X}_t \mathbf{W}_{xh} + \mathbf{H}_{t-1} \mathbf{W}_{hh} + \mathbf{b}_h) \quad (1.5)$$

Trong đó:

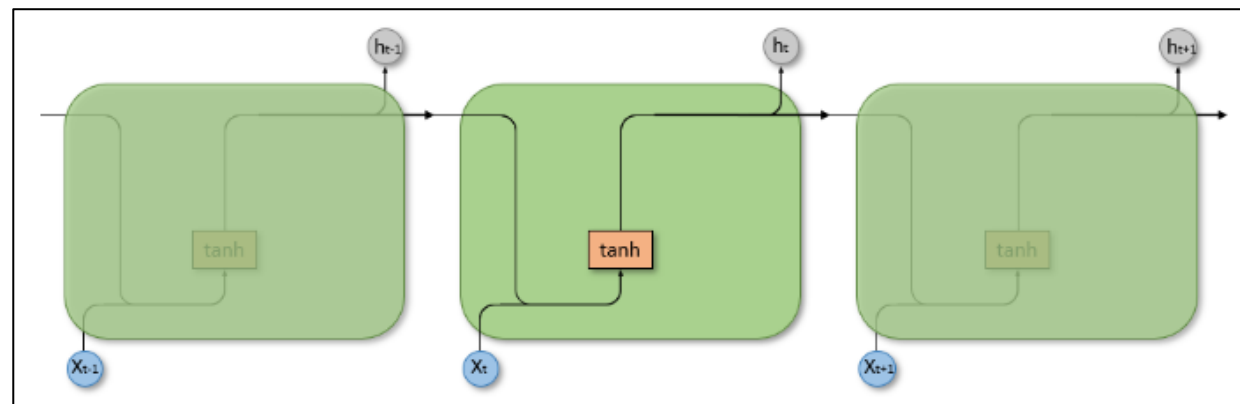
- $\mathbf{X}_t \in \mathbb{R}^{n \times d}$: Input tại thời điểm t (vector hoặc batch).
- $\mathbf{H}_t \in \mathbb{R}^{n \times h}$: Hidden state tại thời điểm t .
- $\mathbf{W}_{xh} \in \mathbb{R}^{d \times h}$: Trọng số từ input đến hidden.
- $\mathbf{W}_{hh} \in \mathbb{R}^{h \times h}$: Trọng số từ hidden trước tới hidden hiện tại.
- $\mathbf{b}_h \in \mathbb{R}^{1 \times h}$: Bias cho hidden state.
- φ : Hàm kích hoạt (thường là tanh hoặc ReLU).

Output sinh ra:

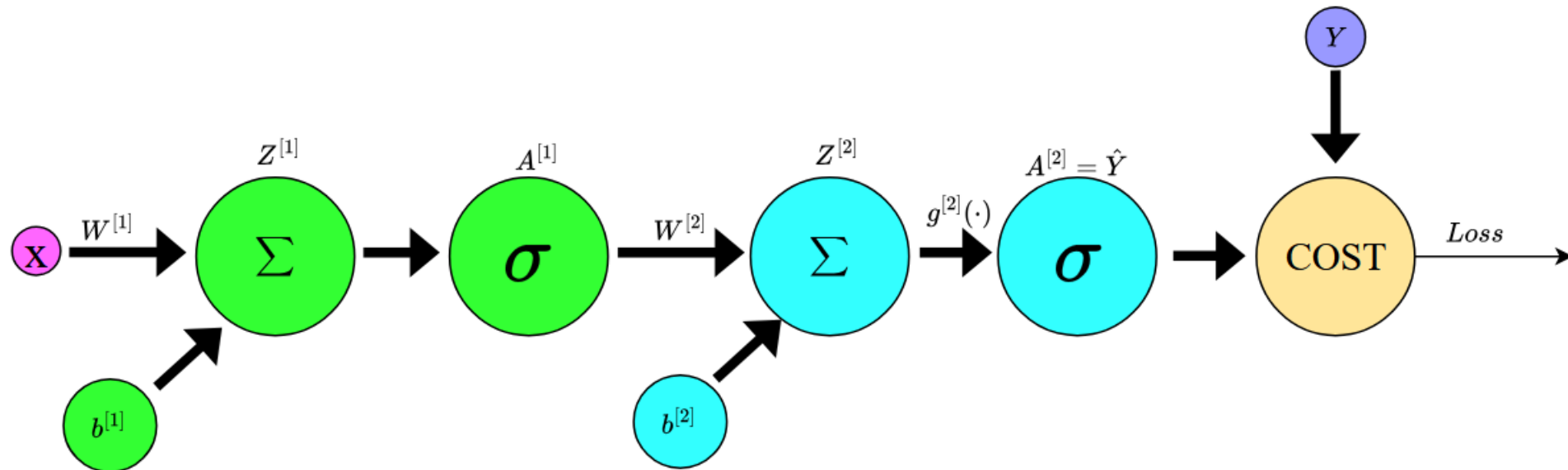
$$\mathbf{O}_t = \mathbf{H}_t \mathbf{W}_{hq} + \mathbf{b}_q \quad (1.6)$$

Trong đó:

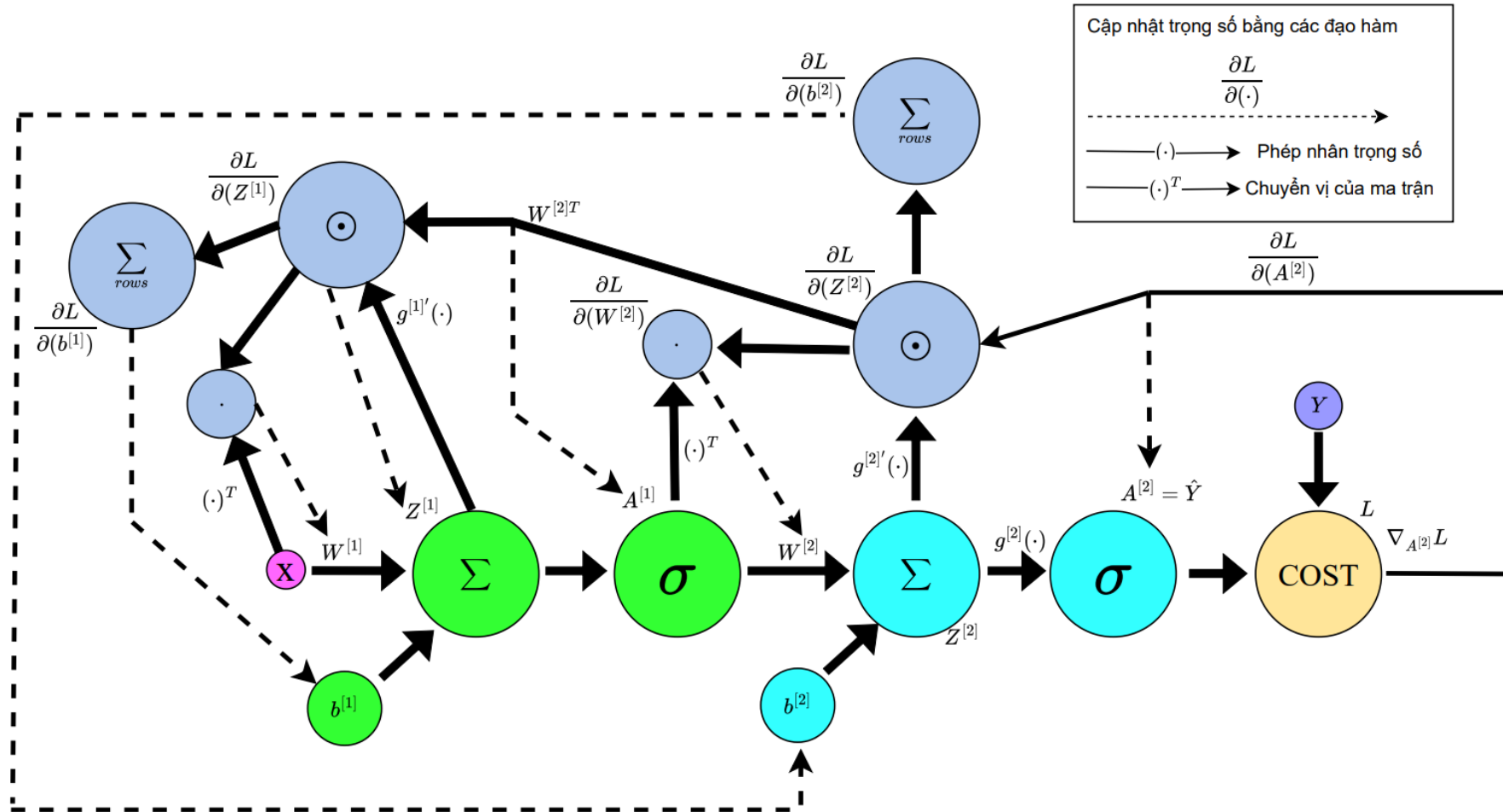
- $\mathbf{O}_t \in \mathbb{R}^{n \times q}$: Output tại thời điểm t .
- $\mathbf{W}_{hq} \in \mathbb{R}^{h \times q}$: Trọng số từ hidden đến output.
- $\mathbf{b}_q \in \mathbb{R}^{1 \times q}$: Bias cho output.



Nền tảng Toán học của Lan truyền ngược qua thời gian

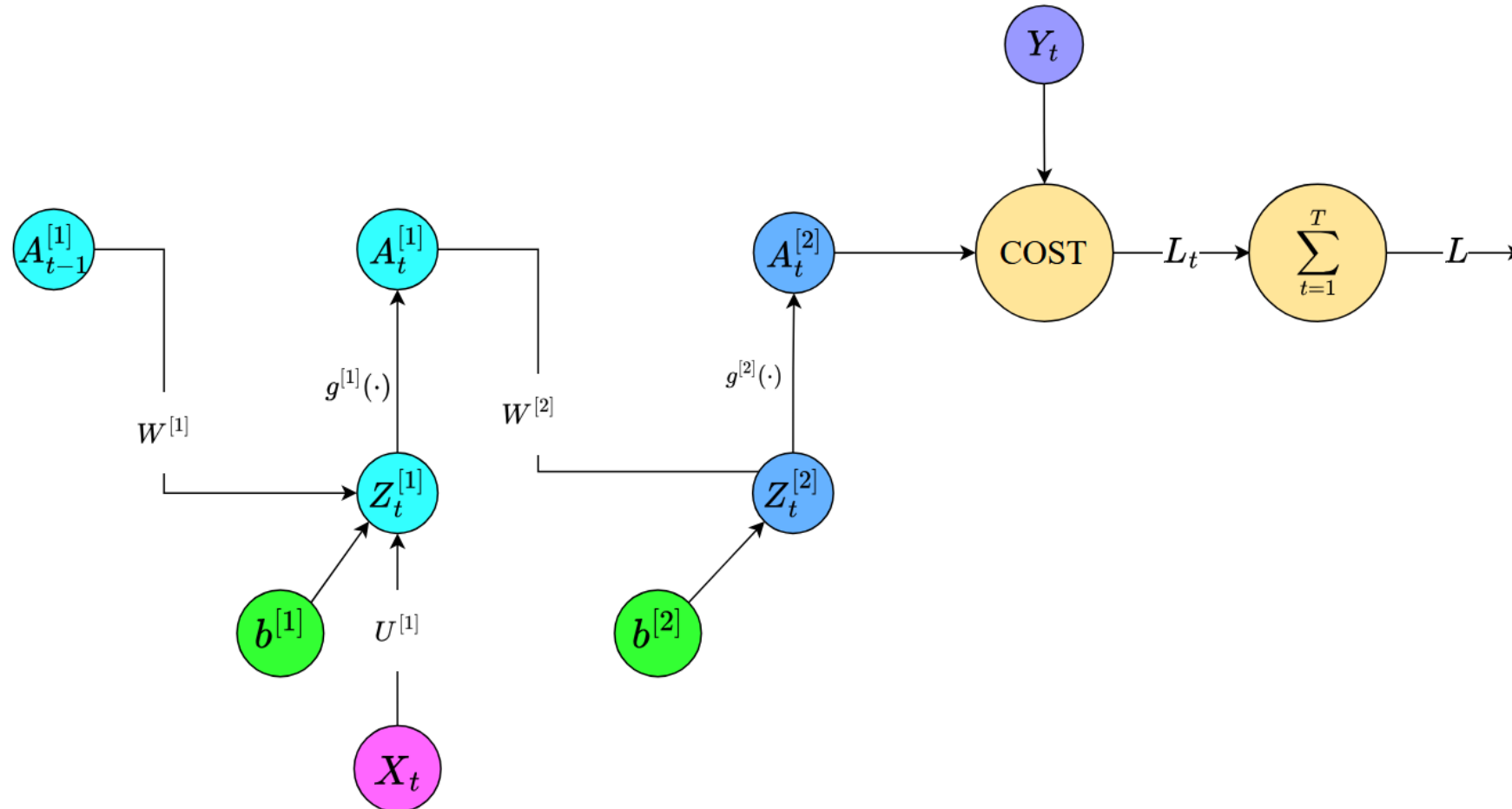


Nền tảng Toán học của Lan truyền ngược qua thời gian



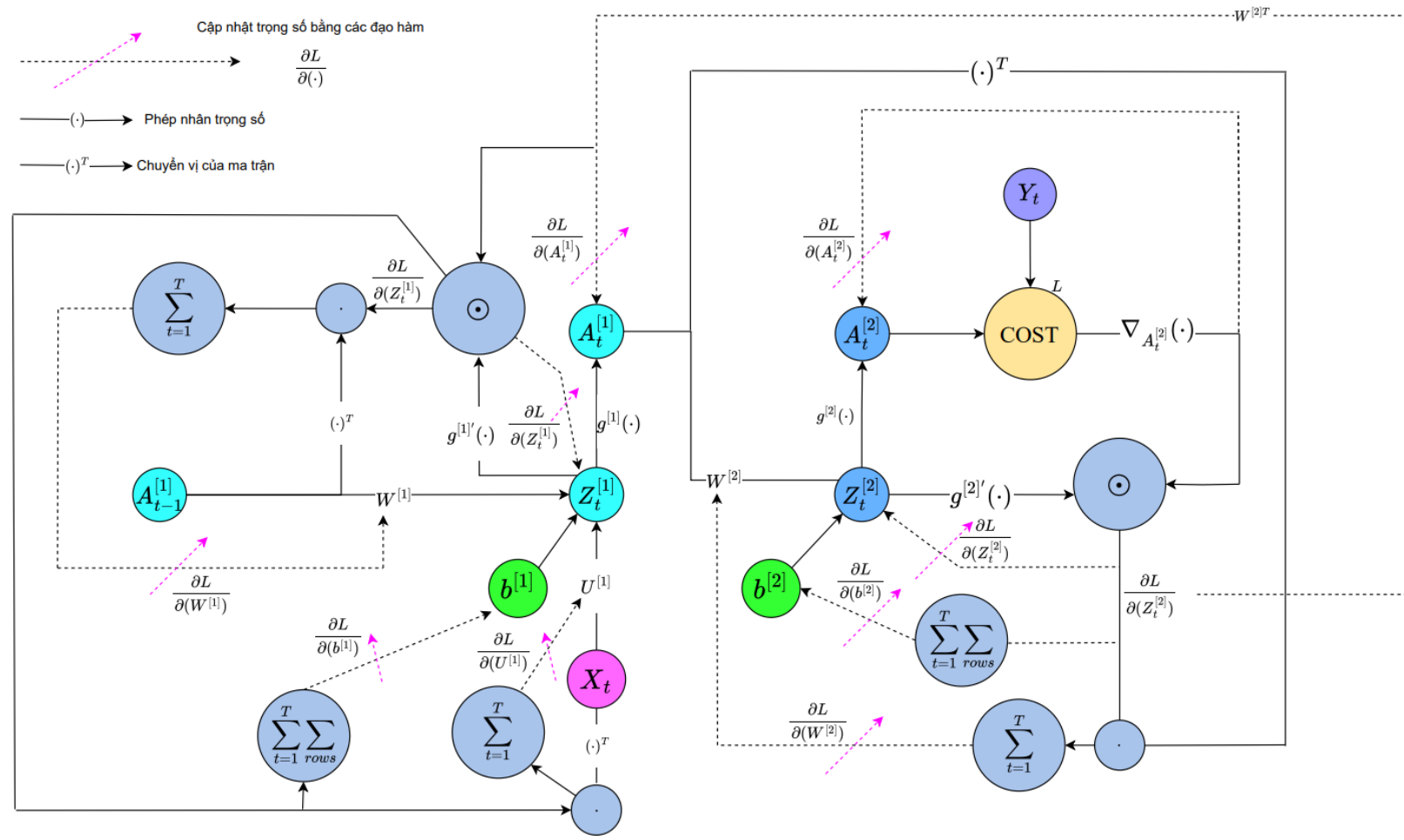
Lan truyền ngược qua thời gian

Nền tảng Toán học của Lan truyền ngược qua thời gian



Lan truyền ngược qua thời gian

Nền tảng Toán học của Lan truyền ngược qua thời gian



Nền tảng Toán học của Lan truyền ngược qua thời gian

Đầu tiên, ta cần tính **đạo hàm của đầu ra** s_t đối với **hàm mất mát** J , được ký hiệu là:

$$\frac{\partial J}{\partial s_t} \quad (2.1)$$

Quan hệ đệ quy để lan truyền gradient theo thời gian trong RNN - sử dụng **quy tắc chuỗi (chain rule)** - được biểu diễn như sau:

$$\frac{\partial J}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot W \quad (2.2)$$

Trong đó: W là ma trận trọng số kết nối giữa trạng thái ẩn hiện tại và trạng thái ẩn trước.

$$\frac{\partial J}{\partial U} = \sum_{t=0}^n \frac{\partial J}{\partial s_t} \cdot x_t \quad (2.3)$$

$$\frac{\partial J}{\partial W} = \sum_{t=0}^n \frac{\partial J}{\partial s_t} \cdot s_{t-1} \quad (2.4)$$

Trong đó:

- x_t là đầu vào tại thời điểm t ,
- s_t là trạng thái ẩn tại thời điểm t ,
- s_{t-1} là trạng thái ẩn tại thời điểm $t - 1$.

Nền tảng Toán học của Lan truyền ngược qua thời gian

Quy tắc cập nhật gradient descent là:

$$\theta := \theta - \eta \cdot \nabla_{\theta} J(\theta) \quad (2.5)$$

Trong đó:

- θ : Các tham số (hoặc vector tham số) mà chúng ta đang cố gắng tối ưu hóa.
- η : Hệ số học (learning rate), một hằng số dương nhỏ.
- $J(\theta)$: Hàm mất mát (hay hàm mục tiêu) mà chúng ta muốn tối thiểu hóa.
- $\nabla_{\theta} J(\theta)$: Gradient của hàm mất mát theo θ .

Ở mỗi bước:

- Tính toán gradient $\nabla_{\theta} J(\theta)$, chỉ ra hướng tăng nhanh nhất.
- Hiệu của gradient này, được nhân với η , để di chuyển theo hướng giảm nhanh nhất (tức là, tối thiểu hóa hàm mục tiêu).

$$U = U - \eta \cdot \frac{\partial L}{\partial U} \quad (2.6)$$

$$W = W - \eta \cdot \frac{\partial L}{\partial W} \quad (2.7)$$

Trong đó:

- U và W là các ma trận trọng số trong RNN,
- $\frac{\partial L}{\partial U}$ và $\frac{\partial L}{\partial W}$ là gradient của hàm mất mát L đối với trọng số U và W tương ứng.

Các vấn đề tiềm ẩn: vanishing/exploding gradients

Nền tảng Toán học của Lan truyền ngược qua thời gian

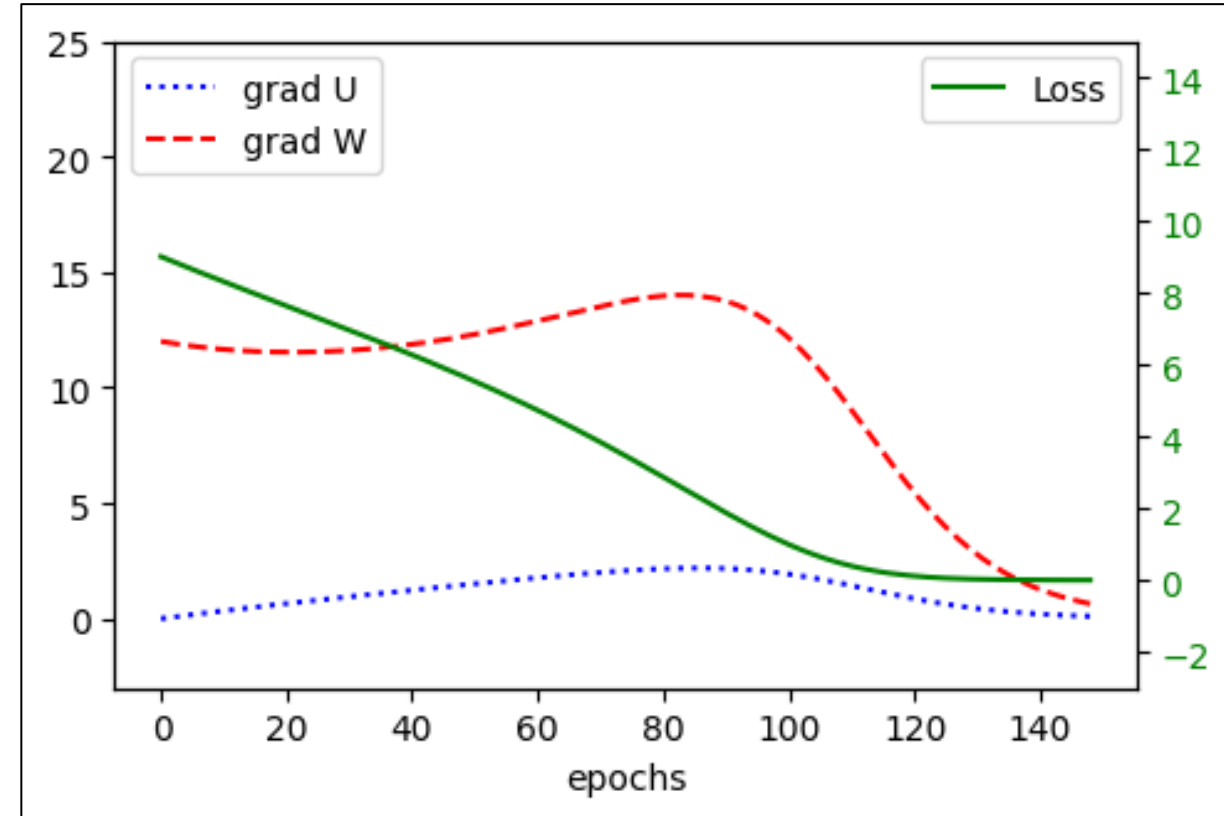
Bài toán đơn giản này giúp minh họa khả năng của RNN trong việc ghi nhớ và xử lý thông tin theo trình tự, cũng như học các phép toán tích lũy cơ bản trên chuỗi.

Mối quan hệ hồi quy được xác định bởi mạng RNN này là:

$$s_t = f(s_{t-1}W + x_tU) \quad (2.8)$$

Để đơn giản hóa, đây là một mô hình tuyến tính và chúng ta không áp dụng hàm phi tuyến trong công thức này. Các trạng thái s_t và các trọng số W và U đều là các giá trị vô hướng, x_t đại diện cho một phần tử trong chuỗi đầu vào (có thể là 0 hoặc 1).

- Nếu đặt $U = 1$, khi có đầu vào, ta sẽ nhận được giá trị đầy đủ của nó.
- Nếu đặt $W = 1$, giá trị ta tích lũy sẽ không bị giảm đi qua thời gian.

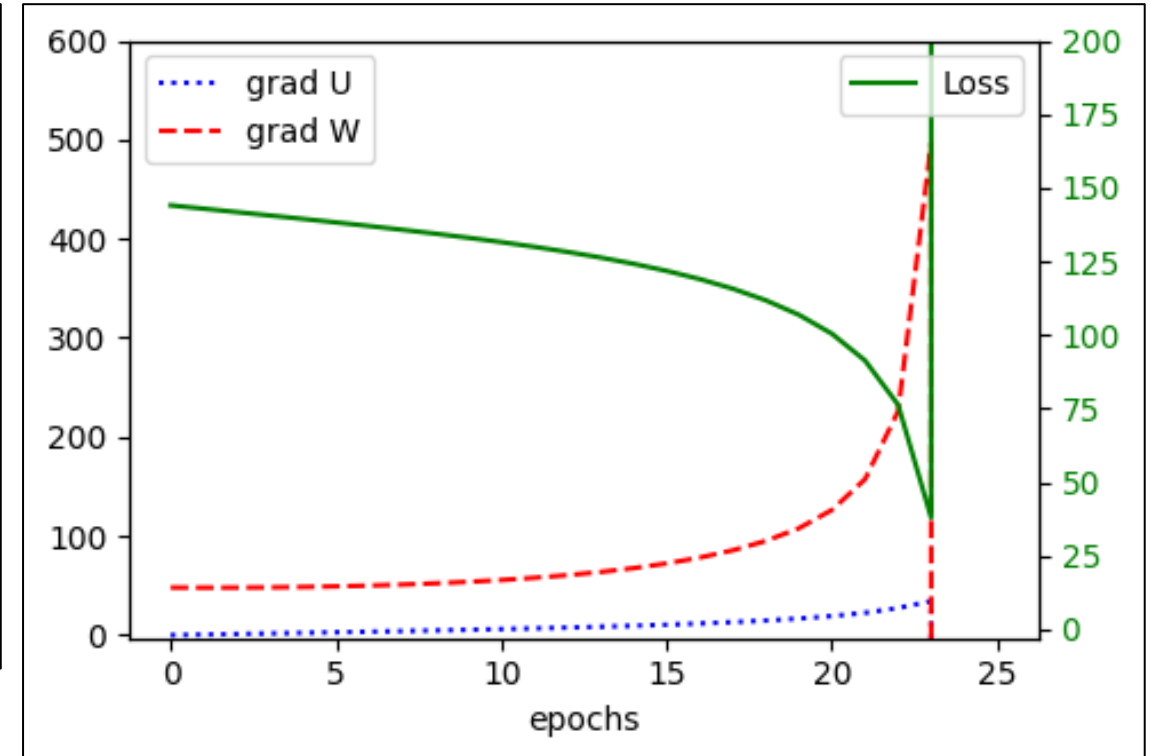


Các vấn đề tiềm ẩn: vanishing/exploding gradients

Nền tảng Toán học của Lan truyền ngược qua thời gian

Sau khi thay đổi thông số huấn luyện với đầu vào là chuỗi nhị phân dài x và đầu ra mong đợi $y = 12$, quá trình huấn luyện diễn ra qua 150 epochs. Tuy nhiên, một hiện tượng đáng chú ý là sự xuất hiện của **bùng nổ gradient** sau khoảng epoch thứ 23, được thể hiện trên hình 2.2. Dưới đây là phân tích chi tiết về hiện tượng này:

- Bùng nổ Gradient:** Trong quá trình huấn luyện, gradient của các trọng số (chẳng hạn như U và W) bắt đầu tăng lên một cách không kiểm soát sau epoch thứ 23. Điều này dẫn đến sự thay đổi mạnh mẽ trong các trọng số, gây ra hiện tượng **bùng nổ gradient**. Hiện tượng này xảy ra khi các giá trị gradient trở nên quá lớn, dẫn đến việc các trọng số bị thay đổi quá mức và không còn ổn định trong quá trình huấn luyện.



Các vấn đề tiềm ẩn: vanishing/exploding gradients

Nền tảng Toán học của Lan truyền ngược qua thời gian

Tính toán Gradient trong RNN

$$\frac{\partial J}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot \frac{\partial s_t}{\partial s_{t-1}} = \frac{\partial J}{\partial s_t} \cdot W \quad (2.9)$$

- Gradient tại một thời điểm phụ thuộc vào tích của nhiều ma trận Jacobian.
- Nếu trị riêng (eigenvalue) của các ma trận này **lớn hơn 1**, tích của chúng sẽ tăng theo cấp số nhân → **bùng nổ gradient**.

Chia sẻ tham số (Parameter Sharing)

- RNN sử dụng **cùng một tập tham số tại mỗi bước thời gian**, nên bất kỳ sự không ổn định nào cũng sẽ được **lặp lại và tích lũy**.
- Trong mạng feedforward, mỗi lớp có bộ tham số riêng biệt, nên nếu một lớp gây ra vấn đề, các lớp khác có thể điều chỉnh tham số riêng để "bù trừ". Đây là một dạng tự điều chỉnh nội bộ giữa các lớp - linh hoạt hơn trong việc xử lý nhiễu cục bộ.
- Vì RNN dùng chung một bộ tham số qua các bước thời gian, nên không có cách nào để "cân bằng lẫn nhau" giữa các bước thời gian bằng việc chỉnh tham số độc lập. Nếu bước thời gian trước gây lỗi, bước sau không thể điều chỉnh tham số riêng để sửa lỗi - nó vẫn dùng cùng một hàm. Vì vậy, so với mạng feedforward, RNN ít linh hoạt hơn theo chiều thời gian.

$$\frac{\partial s_t}{\partial s_{t-k}} = \prod_{j=1}^k \phi'(z_{t-j+1})W = W^k \quad (2.10)$$

Trong một mạng **RNN tuyến tính đơn giản** với trọng số vô hướng W , gradient giữa hai thời điểm cách nhau k bước sẽ cho ra sai khác bằng một đại lượng W^k . Điều này có nghĩa là:

- Nếu $|W| > 1$ thì gradient sẽ **tăng theo cấp số nhân** khi số bước thời gian tăng lên → hiện tượng **bùng nổ gradient**.
- Nếu $|W| < 1$ thì gradient sẽ **giảm theo cấp số nhân** → hiện tượng **tiêu biến gradient**.

Khi W là một **ma trận**, điều này được tổng quát hóa: độ lớn của gradient bị chi phối bởi **bán kính phổ** (*spectral radius*) $\rho(W)$ — tức là trị tuyệt đối lớn nhất trong các trị riêng (eigenvalue) của W :

- Nếu $\rho(W) > 1$, gradient có xu hướng **bùng nổ**.
- Nếu $\rho(W) < 1$, gradient có xu hướng **tiêu biến**.

Tuy nhiên, hành vi cụ thể còn phụ thuộc vào **hàm kích hoạt** được sử dụng và đạo hàm của nó tại mỗi bước thời gian.

Full Computation BPTT

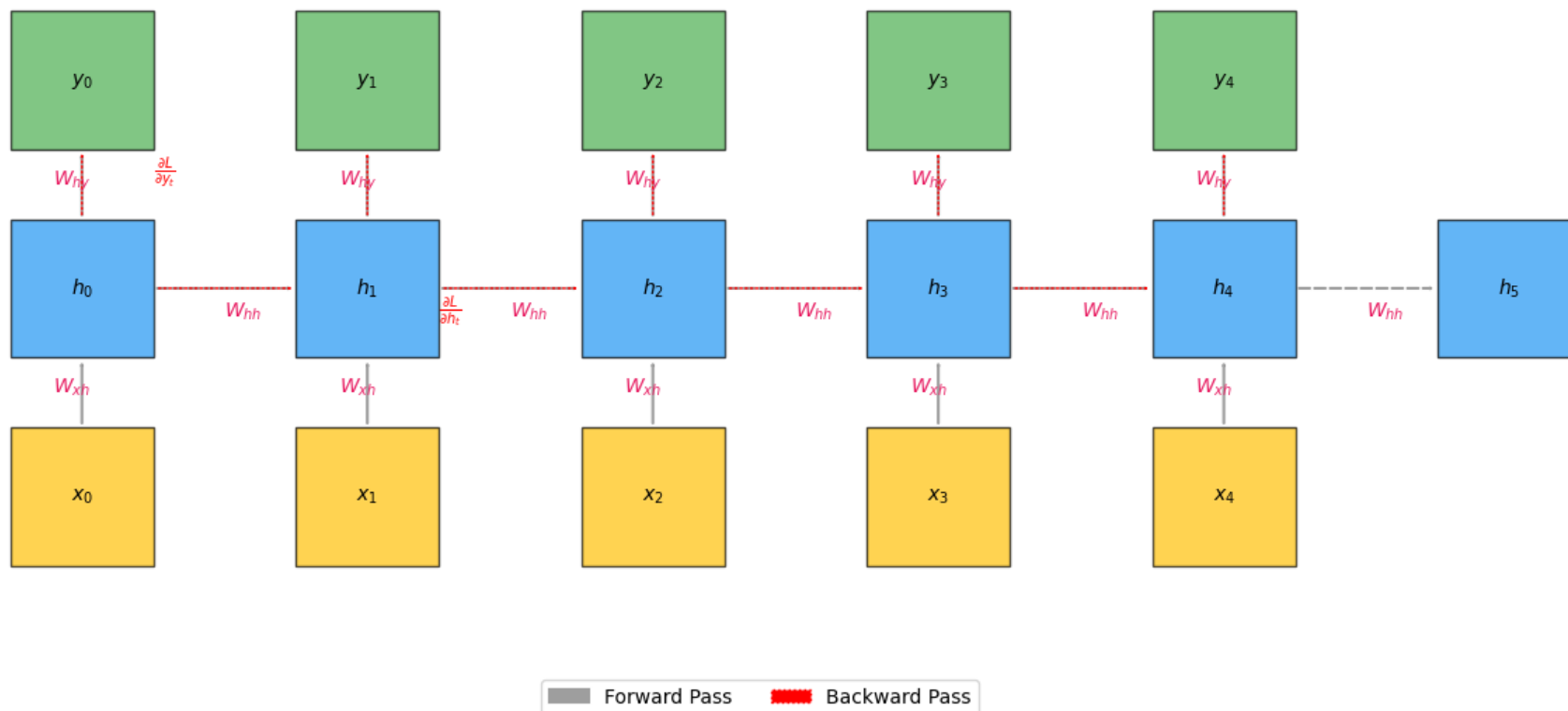
- Tính toán gradient qua TẤT CẢ các bước thời gian
- Lưu trữ tất cả các trạng thái trung gian
- Nắm bắt được các phụ thuộc dài hạn

- Công thức toán học:
$$\frac{\partial h_t}{\partial w_h} = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=1}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}$$

Dòng chảy Gradient trong BPTT

- Bước Forward: lưu trữ tất cả các trạng thái ẩn h_t
- Bước Backward: lan truyền gradient từ cuối ngược về
- Gradient chảy qua hai đường:
 1. Từ output đến hidden state: W_{hy}
 2. Từ hidden state hiện tại đến trước đó: W_{hh}

Full Computation in BPTT: Unfolding RNN Through Time



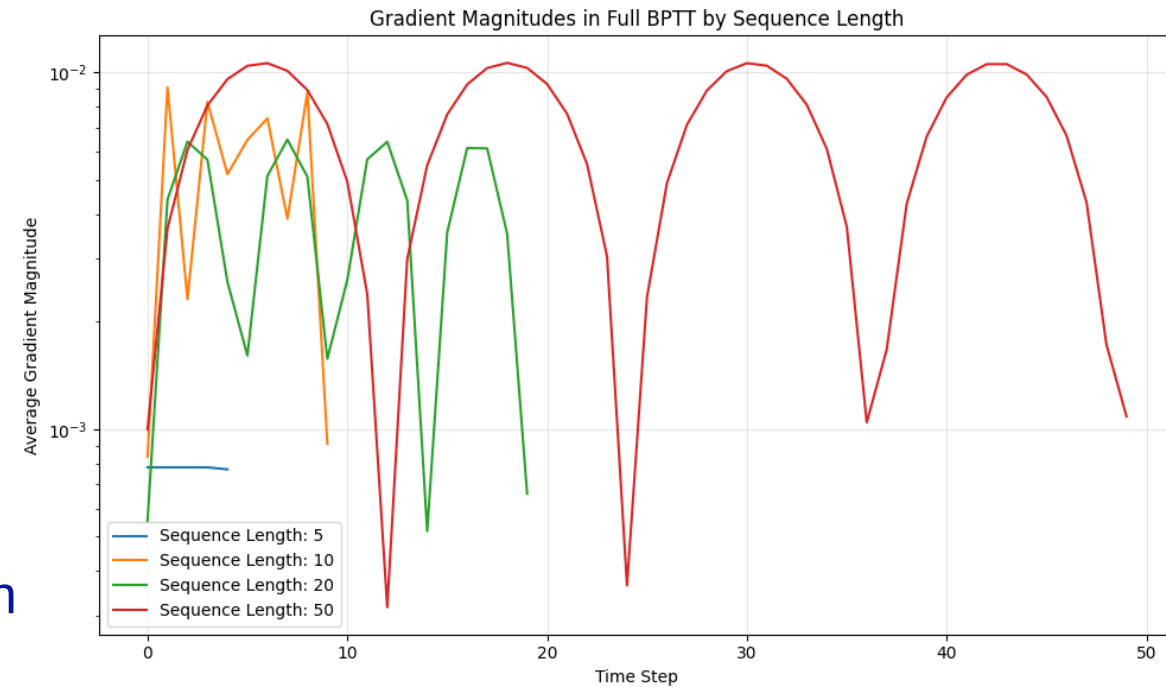
Thách thức của BPTT Đầy đủ

- Tiêu thụ bộ nhớ cao (tỷ lệ thuận với độ dài chuỗi)
- Vấn đề gradient biến mất (vanishing gradient)
- Gradient trở nên cực kỳ nhỏ với chuỗi dài
- Vấn đề gradient bùng nổ (exploding gradient)
- Gradient tăng theo cấp số nhân
- Nhạy cảm với điều kiện ban đầu ("hiệu ứng cánh bướm")



Hành vi của Gradient trong BPTT

- Độ lớn của gradient giảm theo cấp số nhân với độ dài chuỗi.
- Điều này hạn chế khả năng của mạng trong việc học các phụ thuộc dài hạn.
- Phụ thuộc vào hàm kích hoạt (tanh) trong quá trình lan truyền gradient.



Giải pháp và Các phương pháp huấn luyện khác Fixed Truncation

- Ý tưởng: nhằm giải quyết vấn đề vanishing/exploding gradient khi làm việc với chuỗi dài, ta sẽ cắt ngắn quá trình lan truyền ngược bằng cách chia chuỗi đầu vào thành các đoạn con (subsequences) có độ dài cố định.
- $$z_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \sum_{i=\max(1, t-k)}^{t-1} \left(\prod_{j=i+1}^t \frac{\partial f(x_j, h_{j-1}, w_h)}{\partial h_{j-1}} \right) \frac{\partial f(x_i, h_{i-1}, w_h)}{\partial w_h}$$
 - Giới hạn tổng: Tổng Σ chỉ chạy từ $i = \max(1, t - k)$ đến $t - 1$, nghĩa là chỉ xét tối đa k bước trước đó (không vượt quá đầu chuỗi)
 - Tích các gradient: tích Π vẫn giữ nguyên, nhưng chỉ áp dụng cho các i trong k bước bên trong subsequence chứa nó, giảm độ phức tạp và tránh vấn đề vanishing/exploding gradient

Giải pháp và Các phương pháp huấn luyện khác Fixed Truncation

- **Ưu điểm**
 - Giảm nguy cơ gradient vanishing/exploding
 - Tiết kiệm bộ nhớ: chỉ lưu trữ gradient cho k bước gần nhất
 - Tốc độ: giảm độ phức tạp từ $O(t)$ xuống còn $O(k)$
- **Nhược điểm**
 - Mất thông tin dài hạn
 - Phụ thuộc vào độ dài k
 - Khó khăn trong việc tối ưu

Giải pháp và Các phương pháp huấn luyện khác Fixed Truncation

- Bài toán:

Ta xét bài toán dự đoán giá trị tiếp theo trong một chuỗi sin bị nhiễu:

$$y_t = \sin(t) + \eta$$

Ta huấn luyện để dự đoán y_{t+1} dựa trên lịch sử $y_0, \dots, y_t$

- Các giá trị đầu vào:
 - Độ dài chuỗi: 10000
 - Số bước trong 1 đoạn: 200
 - epoch: 30
 - learning rate: 0.01

Fixed Truncation

Fix Truncate BPTT

Epoch 5/30, Loss: 0.065604

Epoch 10/30, Loss: 0.036713

Epoch 15/30, Loss: 0.027478

Epoch 20/30, Loss: 0.022530

Epoch 25/30, Loss: 0.016026

Epoch 30/30, Loss: 0.014625

Full BPTT

Epoch 5/30, Loss: 0.021069

Epoch 10/30, Loss: 0.064891

Epoch 15/30, Loss: 0.029585

Epoch 20/30, Loss: 0.019251

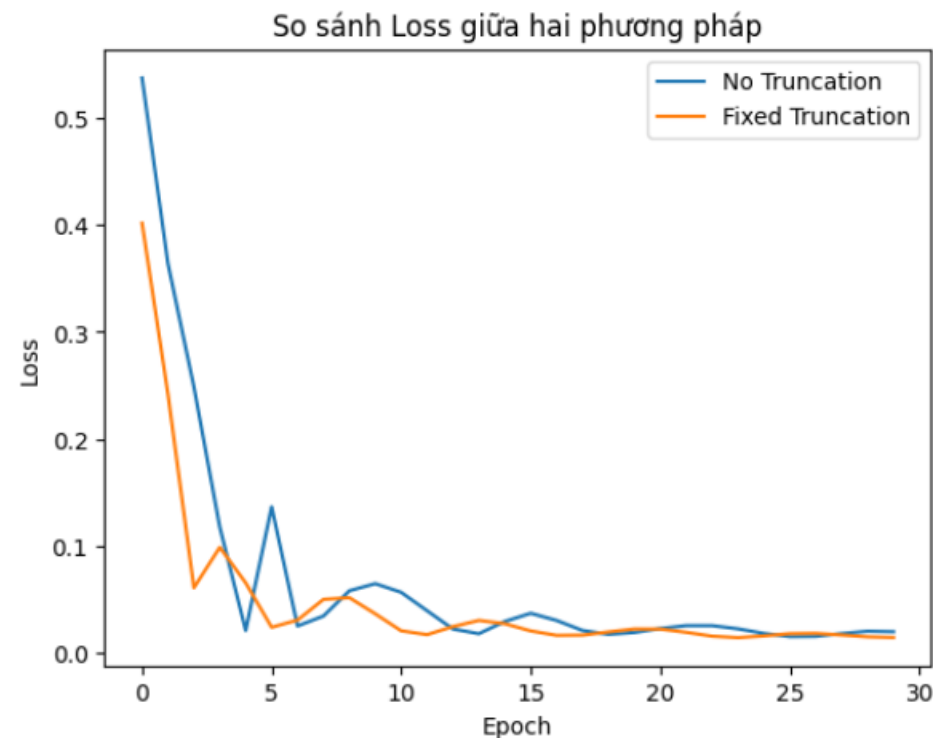
Epoch 25/30, Loss: 0.018388

Epoch 30/30, Loss: 0.019975

==== Kết Quả ====

[No Truncation] Thời gian: 20.93 giây, Bộ nhớ tăng (CPU): 85.87 MB

[Fix Truncation] Thời gian: 20.55 giây, Bộ nhớ tăng (CPU): 0.28 MB



Fixed Truncation

Thử truncate length với các giá trị: 20, 50, 100, 200, 500, 1000, 5000, 100000

==== Summary ====

Method	Time (s)	CPU Mem (MB)
Full BPTT	21.67	85.39
Truncate_20	27.37	0.09
Truncate_50	23.37	0.00
Truncate_100	21.65	0.05
Truncate_200	20.97	0.29
Truncate_500	20.61	0.50
Truncate_1000	20.59	0.61
Truncate_5000	21.59	29.79
Truncate_10000	21.48	85.04

=> Tùy vào yêu cầu bài toán, ta nên thử nghiệm nhiều giá trị k và đánh giá hiệu suất, cân bằng giữa khả năng tính toán và độ chính xác.

Randomized Truncation

- **Ý tưởng:** Thay vì cắt chuỗi tại một điểm cố định, quyết định cắt được thực hiện ngẫu nhiên tại mỗi bước thời gian.

- Biến ngẫu nhiên ξ_t :

- $P(\xi_t = 0) = 1 - \pi_t$

- $P(\xi_t = \pi_t^{-1}) = \pi_t$

- $E[\xi_t] = 1$

- Cập nhật gradient:

$$z_t = \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial w_h} + \xi_t \frac{\partial f(x_t, h_{t-1}, w_h)}{\partial h_{t-1}} \frac{\partial h_{t-1}}{\partial w_h}, \quad E[z_t] = \frac{\partial h_t}{\partial w_h}$$

- Khi $\xi_t = 0$: Dừng lan truyền ngược tại bước t .

Randomized Truncation

- Giảm chi phí: Ít bước lan truyền ngược hơn.
- Ổn định gradient: Tránh gradient quá nhỏ hoặc quá lớn.
- Tăng khả năng tổng quát: Tính ngẫu nhiên giúp mô hình tránh overfitting.
- Giữ nguyên kỳ vọng gradient: Đảm bảo tính đúng đắn của quá trình huấn luyện.
- Dễ triển khai, không cần thay đổi kiến trúc mạng
- Hiệu quả với chuỗi rất dài

Randomized Truncation

- Cắt sớm ngẫu nhiên có thể làm bỏ sót thông tin lâu dài trong chuỗi.
- Tính ngẫu nhiên khiến huấn luyện không ổn định nếu không cố định seed.
- Khó chọn tham số: π_t quá nhỏ $\rightarrow$ học không đủ; π_t quá lớn $\rightarrow$ mất lợi ích giảm chi phí.
- Do cắt ngẫu nhiên, quá trình học có thể mất ổn định và chậm hội tụ.

Randomized Truncation

- Ta xét bài toán dự đoán giá trị tiếp theo trong một chuỗi sin bị nhiễu:

$$y_t = \sin(t) + \eta$$

- Ta huấn luyện để dự đoán y_{t+1} dựa trên lịch sử $y_0, \dots, y_t$

Giải pháp xử lý vanishing/exploding gradients bằng Truncated BPTT

Randomized Truncation

Training WITHOUT Randomized Truncation...

Epoch 5/30, Loss: 0.085890
 Epoch 10/30, Loss: 0.023705
 Epoch 15/30, Loss: 0.030206
 Epoch 20/30, Loss: 0.028792
 Epoch 25/30, Loss: 0.021319
 Epoch 30/30, Loss: 0.019487

Training WITH Randomized Truncation ($\pi=0.9$)...

Epoch 5/30, Loss: 0.190057
 Epoch 10/30, Loss: 0.044903
 Epoch 15/30, Loss: 0.031553
 Epoch 20/30, Loss: 0.028873
 Epoch 25/30, Loss: 0.028075
 Epoch 30/30, Loss: 0.024501

==== Tổng Kết ====

[No Truncation] Thời gian: 24.26 giây, Bộ nhớ tăng (CPU): 229.52 MB

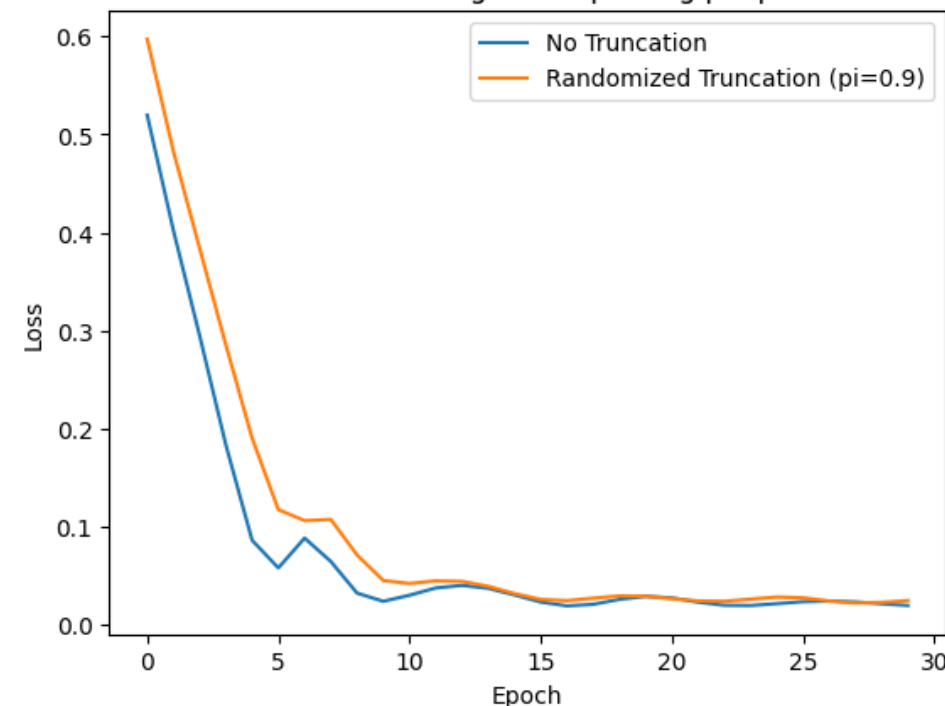
[Random Truncation] Thời gian: 23.98 giây, Bộ nhớ tăng (CPU): 0.62 MB

==== Thống kê GPU (kèm thời gian huấn luyện) ====

[No Truncation] Thời gian: 24.26 giây | Bộ nhớ GPU sử dụng: 16.27 MB | Bộ nhớ GPU reserved: 42.00 MB

[Random Truncation] Thời gian: 23.98 giây | Bộ nhớ GPU sử dụng: 0.02 MB | Bộ nhớ GPU reserved: 0.00 MB

So sánh Loss giữa hai phương pháp



Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.1: So sánh *Full BPTT* và *Truncated BPTT*

Đặc điểm	Full BPTT	Truncated BPTT
Định nghĩa	Lan truyền ngược qua toàn bộ chuỗi	Chỉ lan truyền ngược qua một số bước cố định k
Độ dài chuỗi	Toàn bộ chuỗi (VD: 1000 bước)	Chỉ k bước gần nhất (VD: 20 bước)
Gradient flow	Nhớ được thông tin trong dài hạn	Bộ nhớ ngắn hạn, có thể bỏ lỡ thông tin dài hạn
Chi phí tính toán	Cao	Thấp hơn
Thời gian huấn luyện	Chậm hơn trên mỗi vòng lặp	Nhanh hơn
Phù hợp với	Chuỗi ngắn, cần toàn bộ ngữ cảnh	Chuỗi dài, dữ liệu thời gian thực
Bùng nổ/tiêu biến gradients	Cao (do chuỗi dài)	Thấp hơn, nhưng vẫn tồn tại

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.2: Tổng quát hóa các tình huống sử dụng Full|Truncated BPTT

Tình huống	Phương pháp đề xuất
Tập dữ liệu nhỏ với chuỗi ngắn	BPTT đầy đủ (Full BPTT)
Chuỗi dài (ví dụ: văn bản, âm thanh)	BPTT cắt ngắn (Truncated BPTT)
Ứng dụng có dữ liệu chuỗi thời gian	BPTT cắt ngắn (Truncated BPTT)
Khi việc học các thông tin dài hạn là rất quan trọng	Cân nhắc sử dụng BPTT đầy đủ hoặc LSTM với cửa sổ cắt ngắn dài hơn

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.3: So sánh *Randomized Truncation* và *Standard Truncation*

Đặc điểm	Cắt cố định (Standard)	Cắt ngẫu nhiên (Randomized)
Độ dài cắt	Cố định (VD: 20 bước)	Ngẫu nhiên theo phân phối (VD: phân phối hình học)
Tần suất cập nhật	Đều đặn	Thay đổi qua mỗi vòng
Độ biến thiên gradient	Thấp, cập nhật ổn định	Cao hơn, nhưng trung bình (kỳ vọng) tốt hơn
Học được thông tin dài hạn	Dễ bỏ lỡ do giới hạn k	Linh hoạt, có thể học thông tin dài hạn
Tính toán	Dễ đoán, hiệu quả	Ít đoán trước, khó song song hoá
Trường hợp sử dụng	Huấn luyện ổn định, chuỗi dài cố định	Học online, môi trường stochastic

Giải pháp và Các phương pháp huấn luyện khác

- Gradient Clipping [2]
- BPTT trong LSTM và GRU
- Phương pháp không dùng gradient (Gradient-free methods) [3]
- Real-Time Recurrent Learning (RTRL) [4]

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.5: So sánh luồng gradient trong BPTT của các mô hình

Kiến trúc	Luồng gradient theo thời gian	Đặc điểm chính của BPTT
RNN thường	Gradient được truyền qua cùng một ma trận trọng số lặp lại, dễ gây ra hiện tượng gradient tiêu biến hoặc bùng nổ	Không có cơ chế kiểm soát bộ nhớ, dẫn đến sự mất ổn định
LSTM	Trạng thái ô nhớ giúp duy trì gradient qua thời gian; các cổng điều tiết dòng thông tin, ngăn gradient bị tiêu biến/bùng nổ	Các cổng (đầu vào, quên, đầu ra) kiểm soát luồng gradient và lưu trữ bộ nhớ
GRU	Cổng cập nhật và đặt lại kiểm soát bộ nhớ và dòng thông tin, giúp giảm vấn đề gradient	Cấu trúc cổng đơn giản hơn LSTM nhưng vẫn kiểm soát luồng gradient hiệu quả

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.6: Đánh giá các phương pháp tối ưu phổ biến không dùng gradient

Phương pháp	Mô tả
Random Search	Thử ngẫu nhiên các cấu hình tham số và chọn ra cấu hình tốt nhất. Đơn giản nhưng kém hiệu quả.
Grid Search	Khám phá có hệ thống một lưới tham số. Hiệu quả chỉ khi số chiều thấp.
Thuật toán tiến hóa (Evolutionary Algorithms)	Lấy cảm hứng từ chọn lọc tự nhiên (ví dụ: Genetic Algorithms, Covariance Matrix Adaptation Evolution Strategy (CMA-ES)). Phát triển một quần thể nghiệm.
Tối ưu Bayes (Bayesian Optimization)	Xây dựng mô hình xác suất của hàm mục tiêu và chọn điểm đánh giá dựa trên hàm lợi ích.
Simulated Annealing	Tìm kiếm ngẫu nhiên với “nhiệt độ” giảm dần để điều khiển khả năng chấp nhận nghiệm kém hơn.
Particle Swarm Optimization (PSO)	Phương pháp dựa trên quần thể, nơi các hạt di chuyển trong không gian tìm kiếm, chịu ảnh hưởng bởi nghiệm tốt nhất của chính chúng và của hàng xóm lân cận.

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.7: Tổng quan: BPTT so với RTRL

Đặc điểm	BPTT	RTRL
Phân loại	Học offline / theo lô (batch)	Học online / thời gian thực
Tính toán gradient	Mở rộng mạng theo thời gian rồi lan truyền ngược	Tính gradient đệ quy theo thời gian thực
Bộ nhớ sử dụng	Cao (cần lưu toàn bộ chuỗi để lan truyền ngược)	Rất cao (phải lưu toàn bộ đạo hàm riêng từng phần)
Độ phức tạp tính toán	$O(T \cdot n^2)$	$O(n^4)$ (rất tốn kém)
Phù hợp với chuỗi dài	Có (có thể cắt ngắn)	Không (quá tốn tài nguyên)
Dữ liệu luồng / thời gian thực	Không phù hợp (cập nhật chậm)	Rất phù hợp (cập nhật từng bước thời gian)
Khả năng song song hóa	Dễ dàng (phù hợp với xử lý theo lô)	Khó (cập nhật tuần tự)

Giải pháp và Các phương pháp huấn luyện khác

Bảng 3.8: Các điểm khác biệt chính giữa BPTT và RTRL

Khía cạnh	BPTT	RTRL
Thời gian cập nhật	Cập nhật sau mỗi chuỗi hoặc đoạn (chunk)	Cập nhật ngay tại mỗi bước thời gian
Độ chính xác của gradient	Cắt ngắn, theo batch, hoặc đầy đủ	Chính xác và theo thời gian thực
Hiệu quả tính toán	Hiệu quả nếu dùng cắt ngắn	Không hiệu quả với mô hình lớn
Tính ứng dụng thực tế	Rất phổ biến trong học sâu	Hiếm gặp, chủ yếu trong nghiên cứu

Giải pháp và Các phương pháp huấn luyện khác

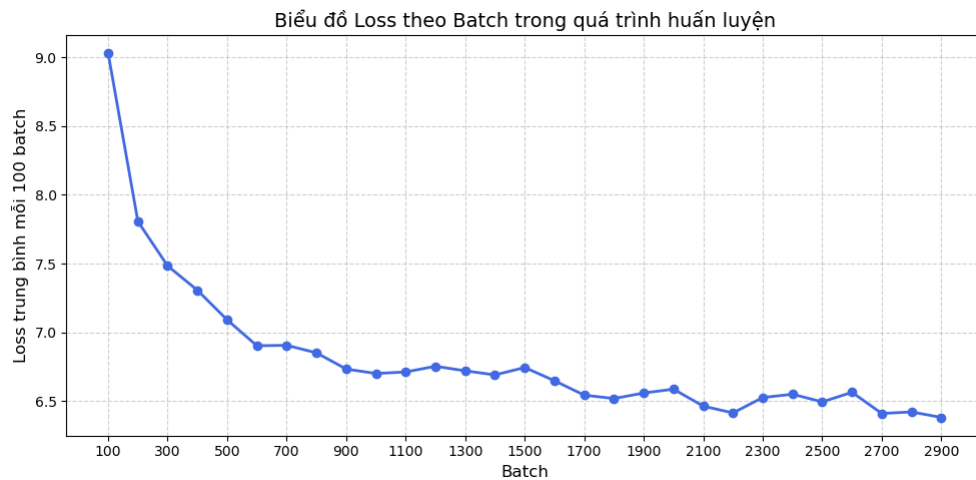
Bảng 3.9: Lựa chọn sử dụng BPTT và RTRL

Tình huống	Phương pháp khuyến nghị
Huấn luyện RNN trên tập dữ liệu lớn	BPTT (có thể cắt ngắn)
Điều khiển thời gian thực hoặc robot học	RTRL (hoặc các biến thể gần đúng như Unbiased Online Recurrent Optimization (UORO))
Dữ liệu luồng hoặc học suốt đời (lifelong learning)	Biến thể của RTRL (nếu tài nguyên tính toán cho phép)

Các ứng dụng của BPTT

- **Xử lý ngôn ngữ tự nhiên (NLP)**
 - Mô hình ngôn ngữ (Language Modeling)
 - Dịch máy (Machine Translation)
 - Nhận dạng giọng nói (Speech Recognition)
- **Dự báo chuỗi thời gian**
 - Dự báo giá cổ phiếu (Stock Price Forecasting)
 - Dự báo thời tiết (Weather Prediction)
- **Robotics**
 - Dự đoán chuỗi hành động (Sequence Prediction)
 - Điều khiển động cơ (Motor Control)
- **Học tăng cường (Reinforcement Learning)**
 - Học từ tín hiệu "thưởng" chuỗi (Learning from Sequential Reward Signals)

BPTT trong Language Modeling



Ngôn ngữ nguồn (English) Ngôn ngữ đích (German)

"I love you"

"Ich liebe dich"

Seed word: Days

Generated text: Days Que for a Nantucket . Alabama State began a drive informed NGC on subjecting 1 @,@ 500 . There = Julien Sun = = Columbus finds 's all films work protection took a one successful good on a Starsmith score stress football than females and to cancel yards to the second and written school in their volume , and therefore

Diễn giải kết quả sinh văn bản (chưa mượt, có từ không logic):

- Do huấn luyện ít (1 epoch).
- Dùng RNN cơ bản (khả năng nhớ dài hạn kém).
- Dấu hiệu mô hình đang học: Cặp từ hợp lý, cụm từ đúng cấu trúc, từ lặp lại hợp ngữ cảnh $\Rightarrow$ Học quy luật ngôn ngữ cơ bản.

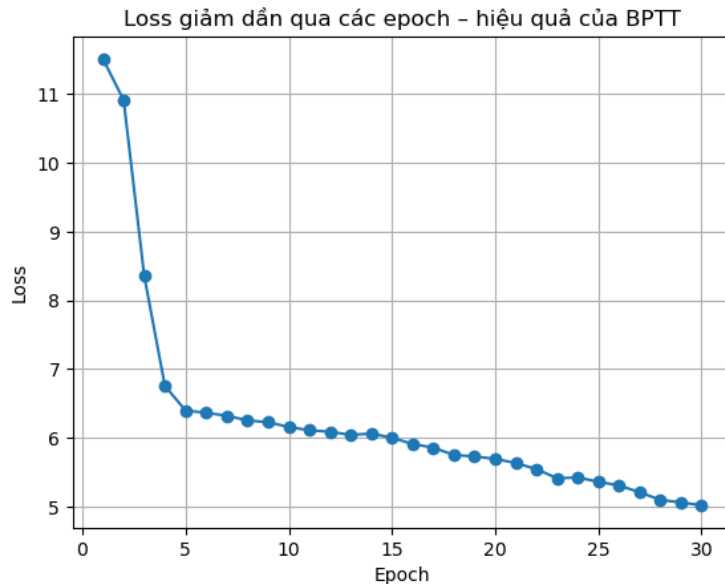
Cải thiện chất lượng:

- Tăng số epoch (ví dụ: lên 50).
- Nâng cấp kiến trúc (ví dụ: dùng LSTM thay RNN).

Kết luận:

- Chất lượng sinh văn bản hạn chế không phủ nhận hiệu quả BPTT.
- Mô hình đã học đặc trưng ngôn ngữ ban đầu, chứng minh BPTT hoạt động.
- Chất lượng sẽ cải thiện khi tăng epoch hoặc dùng LSTM/GRU.

BPTT trong Machine Translation



Quan sát sớm:

- Đầu ra ban đầu có thể sai lệch.
- Dấu hiệu BPTT hoạt động: Cấu trúc câu, từ vựng, thứ tự từ hợp lý.

Kết luận từ trực quan hóa:

- Trực quan hóa đánh giá định tính hiệu quả học/vai trò BPTT.
- Mô hình có xu hướng học cấu trúc nếu huấn luyện đúng.

Kết quả Seq2Seq (30 epoch):

- BPTT hiệu quả: Loss giảm (5.79 → 1.07), học phụ thuộc từ.
- Sinh ngữ pháp đúng dù dữ liệu nhỏ, không attention.
- BPTT cốt lõi: Tối ưu hóa dịch máy Seq2Seq bằng cách truyền lỗi qua chuỗi thời gian.

Tóm lại: BPTT chứng minh hiệu quả trong học dịch máy chuỗi-chuỗi.

Ngôn ngữ nguồn (English)	Ngôn ngữ đích (German)
"I love you"	"Ich liebe dich"

[SRC] There was no possibility of taking a walk that day.

[REF] Es war ganz unmöglich, an diesem Tage einen Spaziergang zu machen.

[PRED] ich die sie nicht und die sie nicht und die sie

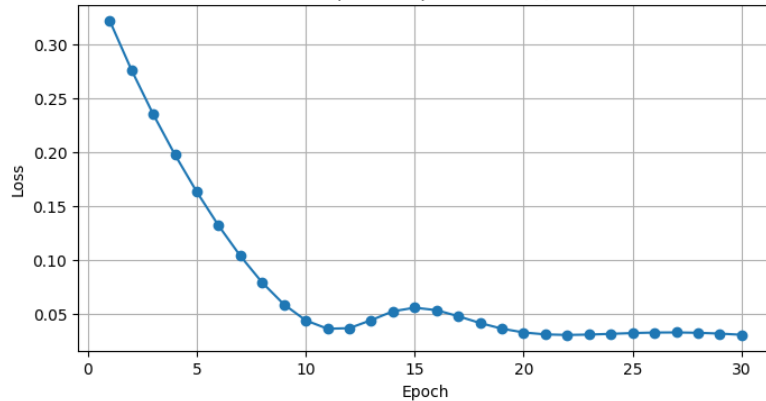
[SRC] The said Eliza, John, and Georgiana were now clustered round their mama in the drawing-room: she lay reclined on a sofa by the fireside, and with her darlings about her (for the time neither quarrelling nor crying) looked perfectly happy.

[REF] Die soeben erwähnten Eliza, John und Georgina hatten sich in diesem Augenblick im Salon um ihre Mama versammelt: diese ruhte auf einem Sofa in der Nähe des Kamins und umgeben von ihren Lieblingen, die zufälligerweise in diesem Moment weder zankten noch schrieten, sah sie vollkommen glücklich aus.

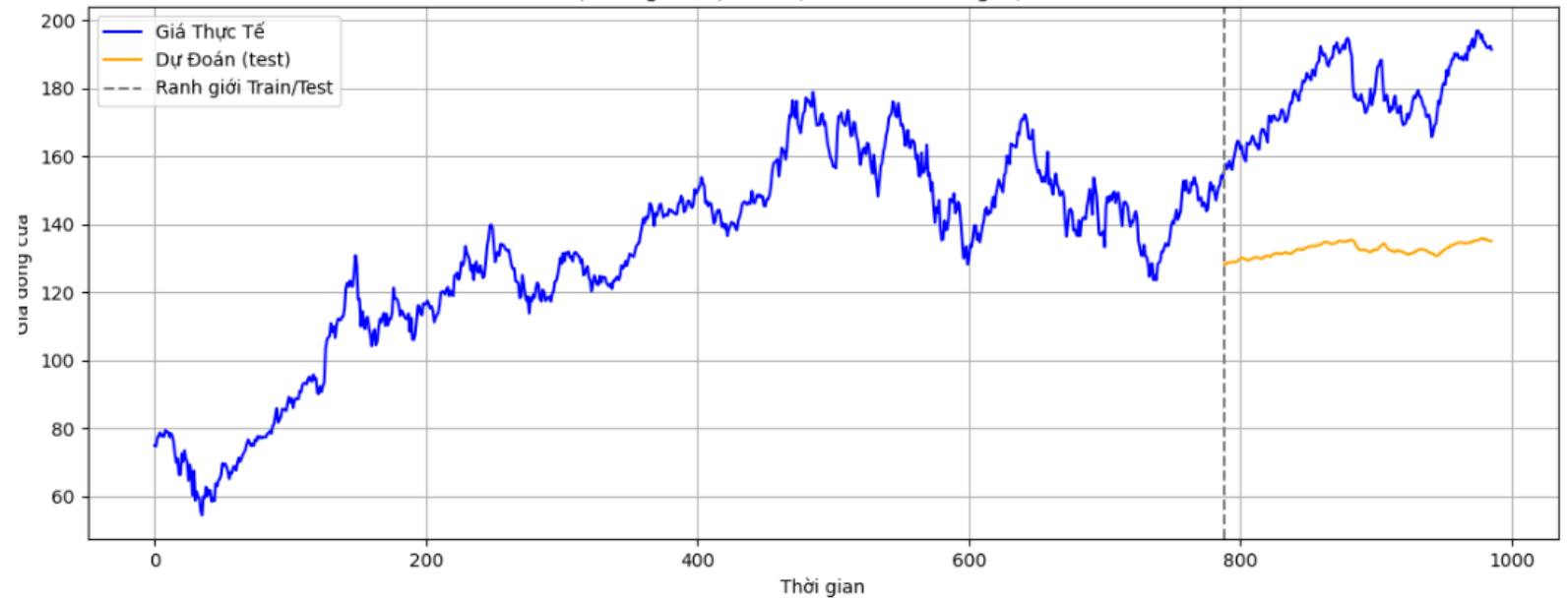
[PRED] ich die sie nicht und die sie nicht und die sie nicht und die und die und die und die

BPTT trong dự báo chuỗi thời gian: Dự báo giá cổ phiếu

Giảm Loss qua các epoch (Minh họa BPTT)



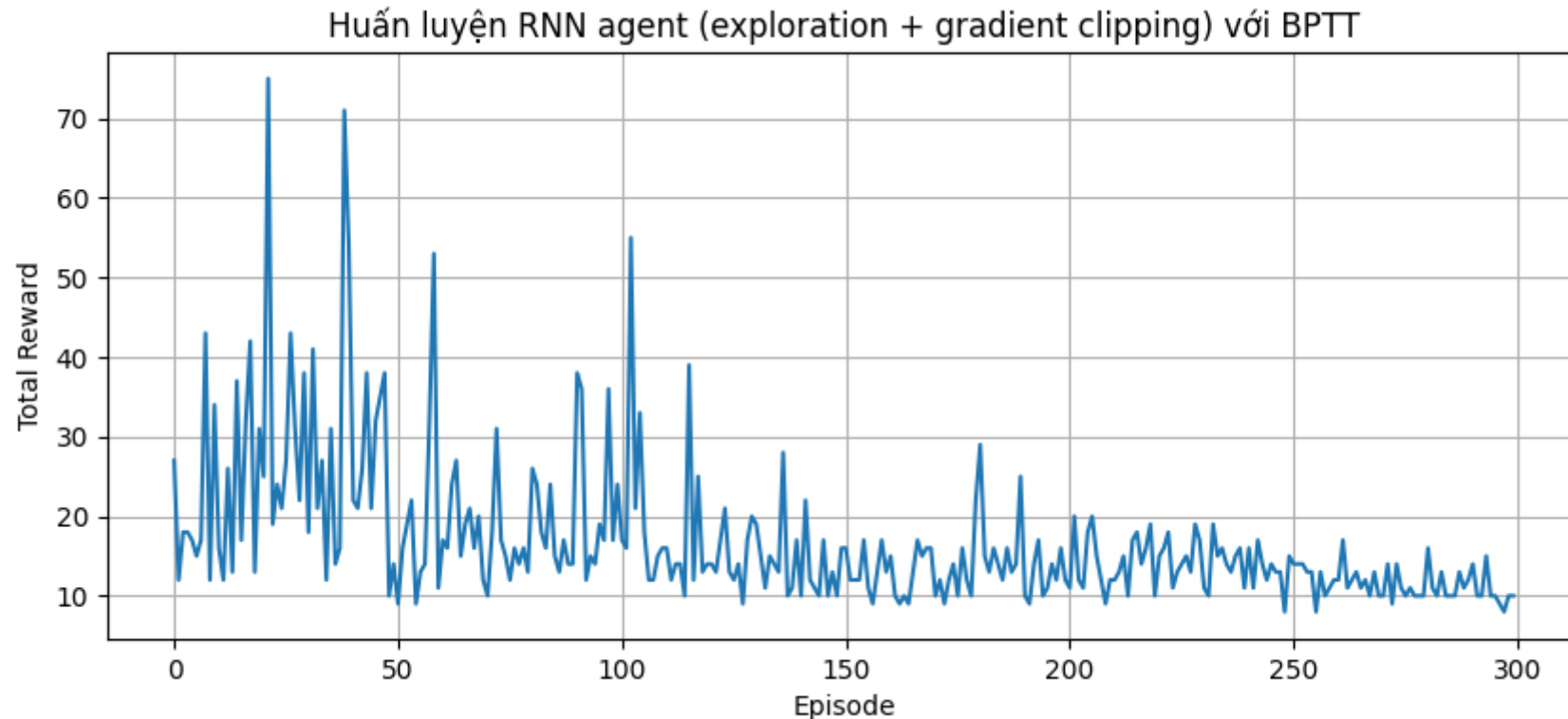
Phân biệt vùng dữ liệu đã học (train) và vùng dự đoán (test)



Kết luận:

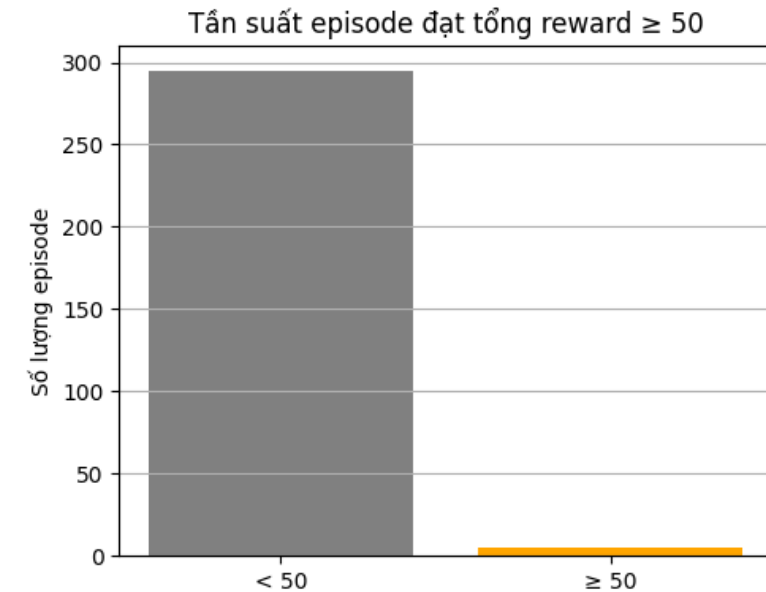
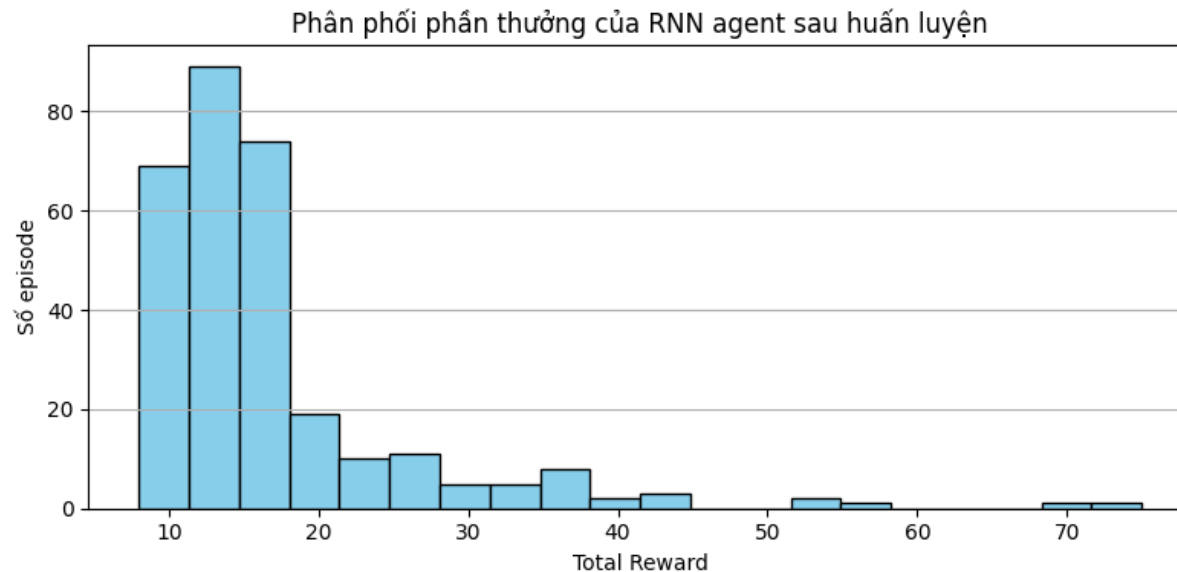
- BPTT là then chốt: Giúp RNN học quy luật biến động giá cổ phiếu theo thời gian.
- Hàm mất mát giảm: Từ 0.41 xuống 0.028 sau 30 epoch, cho thấy tối ưu hóa thành công.
- Dự đoán: Mô hình học được xu hướng trong vùng huấn luyện, nhưng có xu hướng "trơn hóa" ở vùng dự đoán (do RNN đơn giản).
- Vai trò của BPTT: Giúp mô hình nắm bắt động lực cơ bản của dữ liệu chuỗi thời gian bằng cách cập nhật trọng số theo chuỗi phụ thuộc.
- Tầm quan trọng: BPTT là nền tảng cho các mô hình tuần tự (RNN, GRU, LSTM) trong nhiều lĩnh vực (tài chính, y tế, dự báo).
- Hướng phát triển: Sử dụng kiến trúc nâng cao (LSTM, attention...) có thể cải thiện dự đoán, nhưng BPTT vẫn là yếu tố nền tảng.

Biểu đồ huấn luyện RNN agent với BPTT



- Biểu đồ cho thấy reward của agent tăng trong giai đoạn huấn luyện đầu (0-100 episode).
- Tuy nhiên, từ episode 150, hiệu suất trở nên kém ổn định và có xu hướng giảm.
- Điều này gợi ý rằng BPTT giúp RNN nắm bắt được dependencies ngắn hạn, nhưng chưa đủ để học chính sách tối ưu và ổn định trong bài toán CartPole (học tăng cường).
- Việc chỉ dùng RNN đơn giản với loss supervised không đảm bảo xu hướng reward tăng ổn định.

Biểu đồ phần thưởng và tần suất học



- **Biểu đồ Tần suất:** Hầu hết các episode có reward dưới 50, chỉ một số ít đạt được. $\Rightarrow$ Agent học được hành vi hiệu quả trong một số trường hợp, nhưng chưa ổn định.
- **Biểu đồ Phân phối Reward:** Reward tập trung 8-20, nhưng có các điểm rải rác trên 40. $\Rightarrow$ Mô hình học được phản xạ tốt trong ngữ cảnh cụ thể nhờ exploration và BPTT.

Kết luận: BPTT giúp học từ reward tuần tự. Để đạt hiệu suất ổn định và tối ưu, cần:

- Kiến trúc mạnh hơn (LSTM).
- Kỹ thuật discount reward.
- Phương pháp RL chuyên biệt (Policy Gradient).

Hạn chế, Xu hướng nghiên cứu và Định hướng tương lai

- Những hạn chế hiện tại của RNN và BPTT
 - Chi phí tính toán cao
 - Tiêu tốn bộ nhớ
 - Vấn đề gradient
 - Khó song song hóa
- Các hướng tiếp cận thay thế: Cơ chế Attention và Transformer
 - Attention Mechanism [5]
 - Transformer [5]
- Xu hướng nghiên cứu gần đây
 - BPTT + Memory Networks [6]
 - Spiking Neural Networks (SNN) [7]
 - Gradient Approximation [8]
- Định hướng tương lai
 - Neuromorphic Computing [9]
 - Hybrid Architectures [10-15]
 - Continual Learning [16-17]

Tài liệu tham khảo

- [1] Cloudflare, “What is a neural network?” 2025, truy cập vào ngày 04/05/2025. [Online]. Available: <https://www.cloudflare.com/learning/ai/what-is-neural-network/>
- [2] R. Pascanu, T. Mikolov, and Y. Bengio, “On the difficulty of training recurrent neural networks,” ICML, vol. 28, pp. 1310–1318, 2013. [Online]. Available: http://www.icml2013.org/wp-content/uploads/2013/06/icml_2013_paper_519.pdf
- [3] J. Larson, M. Menickelly, and S. M. Wild, “Derivative-free optimization methods,” arXiv preprint arXiv:1904.11585, 2019. [Online]. Available: <https://arxiv.org/abs/1904.11585>
- [4] R. Williams and D. Zipser, “Experimental analysis of the real-time recurrent learning algorithm,” Connection Science, vol. 1, no. 1, pp. 87–111, 1989. [Online]. Available: <https://doi.org/10.1080/09540098908915747>
- [5] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, “Attention is all you need,” in Advances in Neural Information Processing Systems (NeurIPS) 31, 2017, pp. 5998–6008. [Online]. Available: <https://arxiv.org/abs/1706.03762>
- [6] J. Weston, S. Chopra, and A. Bordes, “Memory networks,” arXiv preprint arXiv:1410.3916, 2014. [Online]. Available: <https://arxiv.org/abs/1410.3916>
- [7] K. Yu, V.-K. Vinh-Hieu, D. Bui, and N. Le, “Spiking neural networks and their applications: A review,” Scientific Reports, vol. 11, no. 1, p. 19037, 2021. [Online]. Available: <https://www.nature.com/articles/s41598-021-98448-0>
- [8] M. Boresta, T. Colombo, A. D. Santis, and S. Lucidi, “A mixed finite differences scheme for gradient approximation,” Journal of Optimization Theory and Applications, vol. 194, no. 1, pp. 1–24, 2022. [Online]. Available: <https://doi.org/10.1007/s10957-021-01994-w>
- [9] A. Tuzhilin, “Neuromorphic computing: The next frontier in ai and computing,” University of the People Blog, 2025. [Online]. Available: <https://www.uopeople.edu/blog/neuromorphic-computing/>
- [10] Z. Liu, J. Li, X. Zhang, M. Bansal, and L. Jiang, “Recurrent transformers for sequence modeling,” in Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2021, pp. 4700–4710.
- [11] A. M. Dai, Z. Yang, R. Salakhutdinov, and Q. V. Le, “Transformer-xl: Attentive language models beyond a fixed-length context,” in Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics (ACL), 2019, pp. 2978–2988.

Tài liệu tham khảo

- [12] F. Wu, X. Wu, and X. Sun, “Hybrid rnn-transformer models for sequence-to-sequence learning,” in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 1023–1032.
- [13] H. Liu, J. Zhou, X. Liu, and D. Xu, “Hybrid transformer and rnn models for sequence prediction,” in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 3713–3722.
- [14] H. Yang, F. Zhang, Y. Li, and H. Hu, “Hybrid rnn-transformer networks for time series forecasting,” in Proceedings of the 2021 International Conference on Machine Learning (ICML), 2021, pp. 6235–6245.
- [15] J. Li, L. Wang, H. Zheng, and Y. Liu, “Hybrid rnn-transformer architecture for speech recognition,” in Proceedings of the 2021 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP), 2021, pp. 1632–1636.
- [16] J. Kirkpatrick et al., “Overcoming catastrophic forgetting in neural networks,” Proceedings of the National Academy of Sciences, vol. 114, no. 13, pp. 3521–3526, 2017.
- [17] S.-A. Rebuffi, A. Kolesnikov, and C. H. Lampert, “Learning to learn without forgetting by maximizing transfer and minimizing interference,” in Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2017, pp. 3641–3650.

Nền tảng Toán học của Lan truyền ngược

Hàm lỗi

Hàm lỗi cho ví dụ huấn luyện d được xác định như sau:

$$E_d(\vec{w}) = \frac{1}{2}(t_d - o_d)^2 \quad (6.5)$$

Trong đó, t_d là đầu ra mong muốn, và o_d là đầu ra thực tế của mạng.

Cập nhật trọng số với Gradient Descent

Thuật toán suy giảm độ dốc ngẫu nhiên (Stochastic Gradient Descent) cập nhật các trọng số w_{ji} bằng cách tính đạo hàm hàm lỗi đối với mỗi trọng số và sau đó điều chỉnh chúng. Đối với mỗi ví dụ huấn luyện d , mỗi trọng số w_{ji} được cập nhật theo công thức:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} \quad (6.6)$$

Trong đó, η là hệ số học (learning rate), và hàm lỗi tổng quát E_d được tính bằng tổng lỗi trên tất cả các đơn vị đầu ra.

$$E_d(\vec{w}) \equiv \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 \quad (6.7)$$

Đạo hàm theo trọng số

Để tiếp tục, trọng số w_{ji} có thể ảnh hưởng đến phần còn lại của mạng thông qua net_j . Do đó, chúng ta có thể sử dụng chuỗi đạo hàm (chain rule) để viết:

$$\frac{\partial E_d}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} \frac{\partial net_j}{\partial w_{ji}} = \frac{\partial E_d}{\partial net_j} x_{ji} \quad (6.8)$$

Mục tiêu tiếp theo của chúng ta là tìm một biểu thức thuận tiện cho $\frac{\partial E_d}{\partial net_j}$.

Quy tắc đạo hàm của hàm Lan truyền ngược (Backpropagation)

Nền tảng Toán học của Lan truyền ngược

$$\frac{\partial E_d}{\partial \text{net}_j} = \frac{\partial E_d}{\partial o_j} \frac{\partial o_j}{\partial \text{net}_j} \quad (6.9)$$

Tính toán $\frac{\partial E_d}{\partial o_j}$, chúng ta có:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} \sum_{k \in \text{outputs}} (t_k - o_k)^2 \quad (6.10)$$

Do đạo hàm ở phía bên phải bằng 0 đối với tất cả các đơn vị k khác j , chúng ta có:

$$\frac{\partial E_d}{\partial o_j} = \frac{\partial}{\partial o_j} \frac{1}{2} (t_j - o_j)^2 = -(t_j - o_j) \quad (6.11)$$

Tiếp theo, tính đạo hàm của o_j theo net_j . Đạo hàm này là đạo hàm của hàm sigmoid, do đó:

$$\frac{\partial o_j}{\partial \text{net}_j} = o_j(1 - o_j) \quad (6.12)$$

Kết hợp các biểu thức trên vào 6.9, ta có:

$$\frac{\partial E_d}{\partial \text{net}_j} = -(t_j - o_j) o_j(1 - o_j) \quad (6.13)$$

Kết hợp công thức này với 6.6 và 6.8, ta có quy tắc cập nhật trọng số cho các đơn vị đầu ra:

$$\Delta w_{ji} = -\eta \frac{\partial E_d}{\partial w_{ji}} = \eta (t_j - o_j) o_j(1 - o_j) x_{ji} \quad (6.14)$$

Quy tắc huấn luyện này chính là quy tắc cập nhật trọng số được triển khai trong thuật toán lan truyền ngược.

Quy tắc đạo hàm của hàm Lan truyền ngược (Backpropagation)

Nền tảng Toán học của Lan truyền ngược

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in \text{Downstream}(j)} \frac{\partial E_d}{\partial net_k} \frac{\partial net_k}{\partial net_j} = \sum_{k \in \text{Downstream}(j)} -\delta_k \frac{\partial net_k}{\partial net_j} \quad (6.15)$$

$$= \sum_{k \in \text{Downstream}(j)} -\delta_k \cdot \frac{\partial net_k}{\partial o_j} \cdot \frac{\partial o_j}{\partial net_j} \quad (6.16)$$

Vì $net_k = \sum_j w_{kj} o_j$, nên:

$$\frac{\partial net_k}{\partial o_j} = w_{kj}$$

Và nếu hàm kích hoạt là sigmoid: $o_j = \sigma(net_j)$ thì:

$$\frac{\partial o_j}{\partial net_j} = o_j(1 - o_j)$$

Thay vào biểu thức 6.16, ta được:

$$\frac{\partial E_d}{\partial net_j} = \sum_{k \in \text{Downstream}(j)} -\delta_k w_{kj} o_j(1 - o_j) \quad (6.17)$$

Sắp xếp lại các hạng tử và sử dụng δ_j để ký hiệu cho $-\frac{\partial E_d}{\partial net_j}$, ta có:

$$\delta_j = o_j(1 - o_j) \sum_{k \in \text{Downstream}(j)} \delta_k w_{kj} \quad (6.18)$$

Cuối cùng, quy tắc cập nhật trọng số đối với đơn vị ẩn là:

$$\Delta w_{ji} = \eta \delta_j x_{ji} \quad (6.19)$$

**Nhóm em xin trân trọng cảm ơn Thầy và các bạn đã
lắng nghe**

